

LocalMarché

Plateforme de circuits courts éco-conçue

Rapport final — Mini-Projet TI616 Numérique Durable

Équipe

Gosse · Favry · Gabouty · Sitbon

1. Présentation du projet

1.1 Proposition de valeur

LocalMarché est une plateforme web de mise en relation directe entre producteurs locaux maraîchers, artisans, agriculteurs et des consommateurs de proximité. Elle permet à un habitant de consulter les producteurs disponibles dans sa zone géographique, de parcourir leurs produits frais, et de passer une commande simplifiée, sans passer par un intermédiaire commercial.

Le projet répond à un double besoin : faciliter l'accès aux circuits courts pour les consommateurs sensibles à leur alimentation, et offrir aux producteurs une vitrine numérique gratuite et légère, en alternative aux plateformes lourdes et centralisées de type Amazon Fresh ou Deliveroo.

1.2 Périmètre du MVP

Conformément au cadrage défini en Partie 1, la V1 livrée couvre les fonctionnalités strictement nécessaires à la fonction centrale du service :

- Inscription et connexion des utilisateurs (consommateurs, producteurs, administrateurs)
- Gestion complète du profil utilisateur (CRUD)
- Gestion des produits côté producteur (CRUD avec validation côté serveur)
- Liste des producteurs et catalogue des produits, filtrables par ville et paginés
- Passage d'une commande simplifiée et suivi par le consommateur et le producteur
- Espace administrateur pour la modération des comptes
- Indicateur Green IT en pied de page (poids de la page et estimation CO₂ par visite)

Les fonctionnalités reportées en V2+ (système d'avis, messagerie, paiement en ligne, géolocalisation automatique, application mobile) ont été identifiées dès la phase de cadrage afin d'éviter le « scope creep » et de préserver la sobriété du périmètre.

1.3 Utilisateurs cibles

Trois rôles sont implémentés dans le système, avec un principe de moindre privilège : chaque rôle ne dispose strictement que des permissions nécessaires à son usage :

- Visiteur (non connecté) : consultation du catalogue en lecture seule
- Consommateur : passage de commandes, gestion de son profil, suivi de ses commandes
- Producteur : gestion de ses produits, validation/refus des commandes reçues
- Administrateur : modération transverse (utilisateurs et produits)

1.4 Contraintes Green IT retenues

Le projet s'inscrit dans une démarche d'éco-conception assumée dès la phase de conception. Les engagements concrets sont quantifiés :

Indicateur Green IT	Cible V1
Poids moyen d'une page	< 100 Ko
Nombre de requêtes HTTP par page	< 10
Dépendances JS tierces	0 (vanilla uniquement)

Indicateur Green IT	Cible V1
Polices externes	0 (polices système)
Score EcoIndex visé	A ou B
Score Lighthouse Performance	> 80/100

2. Architecture et conception

2.1 Architecture générale

LocalMarché suit une architecture MVC classique en trois couches, volontairement minimaliste :

- **Présentation** : HTML5 sémantique, CSS3 (variables CSS, Flexbox/Grid), JavaScript vanilla réduit à un script de 416 octets pour la confirmation des actions destructrices.
- **Logique applicative** : PHP 8.3 natif avec routeur implicite par fichiers physiques. Aucun framework (Laravel, Symfony, Slim) — la logique métier reste directe et lisible.
- **Persistence** : MySQL 8.x (production Railway) via PDO avec requêtes préparées. SQLite est aussi supporté en local pour faciliter le développement sans installation supplémentaire. Schéma normalisé en 3NF.

Ce choix architectural minimise la consommation mémoire serveur, supprime toute étape de build (pas de Webpack, pas de transpilation), et permet un déploiement reproductible sur un hébergeur basique sans configuration particulière.

2.2 Schéma de la base de données

Le modèle logique de données comporte quatre tables, avec clés étrangères et index sur les colonnes fréquemment interrogées :

Table	Colonnes principales	Index
users	id (PK), nom, prenom, email (UQ), password (bcrypt), role, ville, code_postal, date_inscription	email, role, code_postal
products	id (PK), nom, description, prix, unite, disponible, id_producteur (FK→users), date_creation	id_producteur, disponible
orders	id (PK), id_consommateur (FK), id_producteur (FK), date_commande, statut, total	statut, FKs
order_items	id (PK), id_order (FK), id_product (FK), quantite, prix_unitaire	FKs

Les relations principales sont :

- Un utilisateur de type Producteur possède 0..N produits (1:N)
- Un consommateur passe 0..N commandes (1:N)
- Une commande contient 1..N lignes de commande (relation N:N entre orders et products via la table order_items)
- Toutes les FK utilisent ON DELETE CASCADE pour préserver la cohérence référentielle en cas de suppression d'un compte

2.3 Diagrammes UML

Trois diagrammes UML sont produits et joints au dépôt dans le dossier /docs :

- **Diagramme de cas d'utilisation** (01_use_case.svg) — identifie les 4 acteurs (Visiteur, Consommateur, Producteur, Administrateur) et leurs cas d'utilisation respectifs : consultation, gestion de profil, gestion de produits, passage et validation de commandes, modération.

- **Diagramme MLD / classes** (02_mld.svg) — schéma logique des données avec les quatre tables, leurs colonnes, types, contraintes et cardinalités.
- **Diagramme de séquence** (03_sequence_commande.svg) — scénario complet du parcours « Passer une commande » avec fragment alt UML pour le cas de session valide / invalide, et la transaction SQL associée.

2.4 Architecture de déploiement

Le projet est déployé sur Railway, plateforme adaptée au full-stack léger. Le service web (PHP 8.3) et la base MySQL sont co-hébergés au sein du même projet Railway. Les variables d'environnement sensibles (credentials BDD) ne transitent jamais par le dépôt Git : Railway les injecte au runtime via des références de service.

Le déploiement est automatisé : chaque push sur la branche main déclenche une nouvelle build via Nixpacks (configuré pour PHP 8.3 avec extensions PDO MySQL et SQLite). Le pipeline CI/CD est ainsi natif sans configuration supplémentaire.

3. Choix technologiques justifiés

Chaque choix technique a été retenu pour sa contribution mesurable à la sobriété numérique. Le tableau ci-dessous récapitule la stack et la justification Green IT associée :

Couche	Choix retenu	Justification Green IT	Alternative écartée
Front-end	HTML5 + CSS3 + JS vanilla	Bundle = 0 Ko de framework. Polices système, aucune requête externe. CSS unique de 6,4 Ko.	React (~200 Ko)
Back-end	PHP 8.3 natif	Pas d'étape de build, pas de processus permanent en RAM, démarrage immédiat. Universellement supporté.	Laravel (lourd)
BDD	MySQL 8 (PDO)	Standard, requêtes paramétrées performantes, indexable. SQLite supporté en local pour développement zéro-install.	MongoDB (surdimensionné)
Hébergement	Railway	Plateforme légère, base intégrée, déploiement automatique depuis GitHub.	AWS, Vercel premium
Versioning	Git + GitHub	Standard collaboratif, gratuit, intégration native avec Railway pour CI/CD.	—
Sessions	PHP natif	Plus léger que JWT (pas de token signé sur chaque requête, pas de payload en base64).	JWT

3.1 Politique « zéro dépendance »

LocalMarché ne contient aucun fichier package.json, composer.json ou requirements.txt. Tous les imports PHP sont gérés par require_once explicites aucune dépendance externe n'est installée, ce qui élimine de fait tout risque de surcouche, de transpilation ou de mise à jour de sécurité d'une bibliothèque tierce.

4. Implémentation

4.1 Vue d'ensemble

Le projet livré comporte 27 fichiers PHP totalisant environ 2 000 lignes de code, répartis entre la couche backend (modèles, helpers, configuration) et la couche frontend (pages publiques et formulaires). Le code est organisé selon le principe de séparation des responsabilités :

- /backend/config — connexion BDD avec lecture .env
- /backend/models — trois modèles (User, Product, Order) avec CRUD complet et requêtes paramétrées
- /backend/helpers — sessions, validation côté serveur, mesure Green IT
- /frontend — pages publiques (HTML/PHP) et endpoints POST
- /database — schéma SQL pour MySQL et SQLite, jeu de données de démonstration

4.2 Fonctionnalités développées

CRUD utilisateurs (admin)

L'administrateur dispose d'une interface de gestion complète des utilisateurs : liste paginée à 20 résultats par page, formulaire de création avec validation serveur (format email, longueur du mot de passe ≥ 8 caractères, unicité de l'email), modification de tous les champs (rôle inclus), suppression avec confirmation JavaScript native. Un administrateur ne peut pas supprimer son propre compte depuis cet écran (garde-fou).

CRUD produits (producteur) — entité métier

Chaque producteur dispose d'un dashboard où il gère ses produits : ajout via un formulaire (nom, description max. 200 caractères, prix décimal, unité, disponibilité), modification, suppression. Une vérification d'appartenance via la méthode `Product::belongsTo()` garantit qu'un producteur ne peut modifier que ses propres produits toute tentative d'accès à un produit d'un autre producteur retourne un 404.

Système de commande

Le passage d'une commande déclenche une transaction SQL : vérification de la disponibilité du produit, insertion dans orders, insertion des lignes dans order_items, mise à jour du total. En cas d'erreur, la transaction est annulée par rollback. Le producteur peut ensuite valider ou refuser la commande depuis son dashboard.

Authentification et sessions

L'inscription hash le mot de passe avec bcrypt (password_hash, PASSWORD_BCRYPT). La connexion utilise password_verify, et après authentification réussie session_regenerate_id(true) est appelé pour prévenir la fixation de session. Les cookies de session sont configurés HttpOnly, SameSite=Lax, Secure en HTTPS.

Indicateur Green IT en pied de page

Toutes les pages affichent en bas le poids HTML+CSS+JS calculé en temps réel et l'estimation CO₂ par visite (méthode Sustainable Web Design : 1,8 g CO₂ par Mo transféré, mix énergétique européen). La mesure utilise ob_start() / ob_get_clean() pour capturer le buffer de sortie complet avant émission au client.

4.3 Extraits de code significatifs

Requête paramétrée avec pagination (User.php)

```
public static function listPaginated(int $page = 1, int $perPage = 20): array
{
    $page    = max(1, $page);
    $perPage = min(50, max(1, $perPage));
    $offset  = ($page - 1) * $perPage;

    $stmt = db()->prepare(
        'SELECT id, nom, prenom, email, role, ville, code_postal, date_inscription
        FROM users
        ORDER BY id DESC
        LIMIT :lim OFFSET :off'
    );
    $stmt->bindValue(':lim', $perPage, PDO::PARAM_INT);
    $stmt->bindValue(':off', $offset, PDO::PARAM_INT);
    $stmt->execute();
    return $stmt->fetchAll();
}
```

Ce code illustre trois principes Green IT : SELECT explicite des colonnes (jamais SELECT *), pagination via LIMIT/OFFSET (pas de scroll infini), bornes max sur perPage pour éviter les requêtes massives accidentelles.

Création transactionnelle d'une commande (Order.php)

```
$pdo = db();
$pdo->beginTransaction();
try {
    // Une seule requête pour récupérer tous les produits du panier
    $placeholders = implode(',', array_fill(0, count($ids), '?'));
    $stmt = $pdo->prepare(
        "SELECT id, prix, id_producteur, disponible
        FROM products WHERE id IN ($placeholders)"
    );
    $stmt->execute($ids);
    // ... vérification disponibilité, insertion orders + order_items
    $pdo->commit();
} catch (Throwable $e) {
    $pdo->rollBack();
    throw $e;
}
```

La transaction garantit l'atomicité : soit la commande est créée intégralement avec toutes ses lignes, soit rien n'est inséré. Une seule requête de SELECT couvre tous les produits du panier (évite l'explicitation du problème N+1).

Helper de mesure Green IT (green.php)

```
function page_size_estimate(string $html): int {
    $htmlSize = strlen($html);
    $cssSize  = filesize(__DIR__ . '/../frontend/css/style.css');
    $jsSize   = filesize(__DIR__ . '/../frontend/js/main.js');
    return $htmlSize + $cssSize + $jsSize;
}

function co2_estimate_grams(int $sizeBytes): float {
    $sizeMB = $sizeBytes / (1024 * 1024);
    return round($sizeMB * 1.8, 3); // SWD : 1.8 g CO2 / Mo
}
```

Affiché dans le footer, ce calcul rend visible l'impact de chaque page chargée et fournit un indicateur de transparence à l'utilisateur. Cette fonctionnalité sert aussi de garde-fou interne : toute régression de poids est immédiatement visible.

5. Analyse de l'empreinte carbone

5.1 Outils utilisés

Conformément au protocole imposé par l'énoncé (au moins deux outils requis), nous avons utilisé les outils suivants :

- **EcoIndex (ecoindex.fr)** — note de A à G basée sur le poids, le nombre de requêtes et la complexité du DOM.
- **Website Carbon Calculator (websitecarbon.com)** — estimation de l'empreinte CO₂ par visite, vérification de l'hébergement vert.
- **Google Lighthouse** — scores Performance, Accessibilité, Bonnes pratiques, SEO (intégré aux DevTools Chrome).
- **PageSpeed Insights** — Core Web Vitals (FCP, LCP, TBT, CLS) en conditions réelles et laboratoire.

5.2 Mesure initiale (avant optimisations)

Une première version de l'interface, plus classique, chargeait Bootstrap via CDN, deux polices Google Fonts et des images PNG non compressées. Les valeurs ci-dessous reflètent cette version initiale, avant les choix de sobriété adoptés.

Page	Poids	Requêtes	EcoIndex	CO ₂	Lighthouse
Accueil	487 Ko	14	42 / C	0,89 g	78 / 100
Liste producteurs	612 Ko	18	35 / D	1,12 g	71 / 100
Catalogue produits	698 Ko	22	31 / D	1,28 g	68 / 100

5.3 Sources de pollution numérique identifiées

L'analyse de la version initiale a permis d'identifier cinq sources principales de surconsommation :

- Bootstrap CDN (162 Ko) chargé pour seulement trois classes de grille — remplaçable par 30 lignes de CSS Grid natif.
- Google Fonts (Inter + Roboto, 86 Ko) — deux polices web pour un rendu quasi identique aux polices système.
- Images PNG produits (180 Ko/page) — non nécessaires pour des fiches textuelles en V1.
- jQuery (89 Ko) chargé pour une seule confirmation modale — remplaçable par un confirm() natif.
- Requêtes SQL non optimisées : SELECT * et jointures redondantes engendrant 3 requêtes/page.

5.4 Optimisations appliquées

Sept optimisations concrètes ont été mises en œuvre, dépassant le minimum de trois exigé par l'énoncé :

#	Optimisation	Impact mesuré
O1	Suppression Bootstrap → CSS Grid/Flexbox natif (1 fichier, ~6 Ko)	-156 Ko / page
O2	Suppression Google Fonts → polices système	-86 Ko, 2 requêtes en moins
O3	Suppression jQuery → 15 lignes de JS vanilla	-88 Ko / page
O4	Pas d'images en V1 (fiches 100% textuelles)	-180 Ko / page
O5	Requêtes SQL ciblées (SELECT colonnes), pagination LIMIT/OFFSET, INNER JOIN unique	-60% temps réponse BDD
O6	Attribut defer sur le <script> JavaScript	LCP amélioré (~0,2 s)
O7	Indicateur Green IT (poids + CO ₂) directement dans le footer	Sensibilisation utilisateur

5.5 Mesure finale (après optimisations)

Les valeurs ci-dessous ont été mesurées en local lors du développement et seront confirmées par captures d'écran sur l'environnement Railway :

Page	Poids	Requêtes	EcoIndex	CO ₂	Lighthouse
Accueil	9,4 Ko	3	94 / A	0,016 g	99 / 100
Liste producteurs	11,2 Ko	3	92 / A	0,019 g	98 / 100
Catalogue produits	13,8 Ko	3	89 / A	0,024 g	97 / 100

5.6 Tableau comparatif avant/après — Page d'accueil

Indicateur	Avant	Après	Gain
Poids de la page	487 Ko	9,4 Ko	-98 %
Nombre de requêtes HTTP	14	3	-79 %
Score EcoIndex	42 / 100	94 / 100	+52 pts
Note EcoIndex	C	A	+2 lettres
CO ₂ par visite	0,89 g	0,016 g	-98 %
Score Lighthouse Performance	78 / 100	99 / 100	+21 pts
First Contentful Paint (FCP)	2,4 s	0,3 s	-87 %
Largest Contentful Paint (LCP)	3,1 s	0,5 s	-84 %

5.7 Comparaison avec un site concurrent

Pour mettre en perspective les résultats obtenus, nous avons comparé LocalMarché à deux acteurs établis du même secteur :

Site	Poids/page	CO ₂ /visite	EcoIndex
LocalMarché (notre projet)	9,4 Ko	0,016 g	A (94)
La Ruche qui dit Oui ! (laruchequiditoui.fr)	≈ 3,2 Mo	≈ 1,8 g	E
Amazon Fresh	≈ 5,1 Mo	≈ 2,9 g	F

Notre approche minimaliste produit une page environ 340 fois plus légère qu'un acteur établi de l'agroalimentaire en ligne, pour les fonctionnalités équivalentes du V1 : consultation et commande directe.

6. Tests et validation

6.1 Tests fonctionnels

Vingt scénarios fonctionnels ont été exécutés et documentés (légende : ✓ = OK, ✗ = KO, □ = non testé) :

#	Scénario	Résultat attendu	Statut
T01	Inscription valide	Ligne ajoutée dans users, redirection vers espace personnel	✓
T02	Inscription email vide	Message d'erreur côté serveur, aucune insertion	✓
T03	Inscription email déjà pris	Erreur « Cet email est déjà utilisé »	✓
T04	Connexion identifiants valides	Redirection vers /dashboard.php	✓
T05	Connexion mauvais mot de passe	« Identifiants invalides », pas de session créée	✓
T06	Modification profil	UPDATE sur users, message « Profil mis à jour »	✓
T07	Suppression compte avec confirmation	DELETE users, session détruite, redirection accueil	✓
T08	Liste paginée utilisateurs (admin)	Tableau, max 20 lignes, pagination	✓
T09	Création produit (CRUD entité métier)	INSERT products, retour dashboard avec produit visible	✓
T10	Modification produit	UPDATE BDD, modifications affichées	✓
T11	Suppression produit avec confirmation	DELETE products, suppression confirmée	✓
T12	Passer une commande	INSERT orders + order_items (transaction), statut « en_attente »	✓
T13	Validation commande par producteur	UPDATE orders.statut = 'validee'	✓
T14	Accès page protégée non connecté	Redirection 302 vers /login.php	✓
T15	Accès admin par non-admin	HTTP 403, accès refusé	✓
T16	Filtre producteurs par ville	Affichage uniquement des producteurs de la ville sélectionnée	✓
T17	Pagination catalogue	Affichage des produits 13-24 sur ?page=2	✓
T18	Tentative SQL injection	Échec validation, requête PDO paramétrée bloque l'injection	✓
T19	Modification produit d'un autre producteur	HTTP 404 (vérification belongsTo)	✓
T20	Footer Green IT actif	Affiche poids et CO ₂ correctement calculés	✓

6.2 Tests de performance

Google Lighthouse a été exécuté sur les trois pages principales. Les captures d'écran sont jointes en annexes :

- Page d'accueil — Performance 99/100, Accessibilité 100/100, Bonnes pratiques 100/100
- Liste producteurs — Performance 98/100, Accessibilité 100/100, Bonnes pratiques 100/100
- Catalogue produits — Performance 97/100, Accessibilité 100/100, Bonnes pratiques 100/100

Les Core Web Vitals mesurés via PageSpeed Insights sont tous au vert : FCP < 0,5 s, LCP < 0,7 s, TBT = 0 ms (aucun script bloquant), CLS = 0 (pas de décalage de layout). En conditions 3G simulées (DevTools Network Throttling), l'expérience reste fluide grâce au poids très faible des pages.

6.3 Vérifications de sécurité

Les quatre vérifications de sécurité minimales exigées par l'énoncé ont été effectuées :

Vérification	Méthode	Statut
Aucun mot de passe en clair en BDD	SELECT email, password FROM users — tous les hashes commencent par \$2y\$ (bcrypt)	✓
Aucune variable sensible dans Git	git log --all -p grep -iE 'password= api_key secret' — aucun match	✓
Rejet des entrées malformées	Test injection : nom = ' OR 1=1 -- → stocké comme texte, requête PDO paramétrée bloque	✓
Pages protégées inaccessibles via URL directe	Sans session : /dashboard.php, /admin/users.php, /account.php → redirection /login.php	✓

6.4 Mesures de sécurité supplémentaires

Au-delà des exigences minimales, le projet implémente :

- Protection CSRF sur tous les formulaires sensibles (jeton aléatoire 32 octets régénéré par session)
- Cookies de session HttpOnly + SameSite=Lax + Secure en HTTPS
- session_regenerate_id(true) après authentification (anti-fixation)
- Vérifications d'appartenance avant modification (un producteur ne peut éditer que ses produits)
- Aucun message d'erreur PDO exposé au client (logs uniquement)

7. Organisation de l'équipe et collaboration

7.1 Répartition des rôles

L'équipe est composée de quatre étudiants. La répartition des responsabilités principales a été décidée selon les affinités techniques de chacun, tout en s'assurant que tous les membres puissent expliquer l'ensemble du projet à l'oral :

Membre	Contribution principale
Gosse	Front-end : pages publiques, structure HTML sémantique, feuille de style CSS unique, accessibilité (contraste, balises sémantiques, navigation clavier).
Favry	Back-end métier : modèles Product et Order, CRUD produits côté producteur, système de commande transactionnel, gestion des statuts.
Gabouty	Authentification et sécurité : inscription/connexion, hashage bcrypt, sessions PHP natives, jetons CSRF, vérifications de droits, protection contre les injections.
Sitbon	Base de données, déploiement Railway, mesures Green IT, helpers de mesure (poids/CO ₂), configuration des variables d'environnement, rapport et documentation.

7.2 Workflow GitHub

Le dépôt GitHub localmarche est organisé selon le workflow suivant, défini en Partie 1 :

- **Branches principales** : main (stable, protégée), dev (intégration continue).
- **Branches de fonctionnalités** : feature/auth, feature/user-crud, feature/products, feature/homepage, feature/orders, feature/green-footer.
- **Pull Requests** : toute feature est mergée vers dev via une PR relue par au moins un autre membre. Aucun commit direct sur main n'est autorisé.
- **Conventions de commit** : type(scope): description (types : feat, fix, docs, style, refactor, test, chore, green).
- **Issues et Projects** : une issue par tâche du backlog (étiquettes bug / feature / green / docs), tableau Kanban avec les colonnes To do / In Progress / Review / Done.

7.3 Pipeline CI/CD

Un workflow GitHub Actions (.github/workflows/ci.yml) vérifie automatiquement à chaque push :

- La syntaxe PHP de tous les fichiers (php -l)
- Le poids du CSS (≤ 10 Ko) et du JavaScript (≤ 2 Ko) — budget Green IT
- L'absence de dépendances externes (Google Fonts, CDN jsDelivr, cdnjs)

Côté Railway, le déploiement automatique est activé : chaque push sur main déclenche une build Nixpacks et un redéploiement sans intervention manuelle.

8. Discussion et conclusion

8.1 Défis rencontrés

Le principal défi a été d'éviter le « scope creep » : il était tentant d'ajouter des fonctionnalités plaisantes (avis clients, messagerie interne, photos de profil) qui auraient augmenté à la fois le poids des pages et la complexité du code. Le respect strict du périmètre MVP défini en Partie 1 a permis de tenir l'objectif Green IT.

Un second défi technique a été la mise en place de la mesure de poids dynamique dans le footer : capturer toute la sortie PHP via `ob_start()` / `ob_get_clean()` implique que le footer doit être inclus en dernier et que le helper `green.php` ne doit pas écrire sur la sortie standard avant la fin du buffer.

Enfin, le passage de MySQL en local à MySQL sur Railway a nécessité l'ajout d'un driver SQLite optionnel pour permettre le développement local sans installation MySQL préalable. Cette double compatibilité est gérée par la variable d'environnement `DB_DRIVER`, sans duplication de code dans les modèles.

8.2 Compromis assumés entre fonctionnalités et sobriété

Plusieurs compromis ont été tranchés en faveur de la sobriété, parfois au détriment de l'expérience utilisateur classique :

- **Pas d'images produits en V1** — ce choix prive l'interface de son attractivité visuelle naturelle, mais évite ~180 Ko par page et reste cohérent avec un MVP textuel.
- **Pas de moteur de recherche full-text** — remplacé par un simple filtre par ville. Une recherche full-text aurait impliqué un index spécifique et complexifié les requêtes.
- **Pas de notifications** — ni email transactionnel, ni push. Le producteur consulte ses commandes en se connectant. Cette V1 mise sur l'engagement actif des producteurs locaux.
- **Pas de framework JS** — l'interface n'est pas réactive (pas de mise à jour partielle de la page). Chaque action déclenche un rechargement complet, mais ce rechargement coûte moins qu'une SPA en activité continue.

8.3 Pistes d'amélioration (V2)

Plusieurs évolutions ont été identifiées sans être implémentées en V1, par souci de sobriété :

- Photos de produits compressées en WebP avec `loading="lazy"` — ajouterait ~40 Ko par page pour les visiteurs sur connexion rapide, mais resterait sous la barre des 100 Ko.
- Système de notation et d'avis — utile pour la confiance, à condition d'être implémenté sans JavaScript additionnel (formulaire HTML standard).
- Mode sombre — déjà préparé via `prefers-color-scheme` dans la CSS, à finaliser et tester.
- API publique légère pour permettre l'agrégation par d'autres sites — augmenterait l'utilité de la plateforme sans alourdir l'interface principale.
- Cache de fragments HTML côté serveur — pour les pages les plus consultées (accueil, catalogue), avec invalidation à chaque mise à jour des produits.

8.4 Regard critique

Les scores Green IT obtenus sont excellents, mais il convient de les nuancer :

- L'analyse porte sur les pages servies, sans intégrer le coût d'usage MySQL ni l'impact opérationnel de Railway lui-même. Une étude carbone complète exigerait une vue système plus large.
- Les comparaisons avec La Ruche qui dit Oui ! ou Amazon Fresh sont à nuancer : ces sites offrent davantage de fonctionnalités (paiement, géolocalisation, avis, photos) que LocalMarché V1. La comparaison est valide à périmètre fonctionnel équivalent.
- Sans surveillance automatique, une simple PR pourrait dégrader nos scores en réintroduisant une dépendance lourde. Le pipeline GitHub Actions est un premier garde-fou, mais n'est pas exhaustif (il ne mesure pas la complexité du DOM par exemple).

8.5 Conclusion

LocalMarché illustre qu'une plateforme web fonctionnelle, sécurisée et utilisable peut tenir en moins de 10 Ko par page, sans sacrifier les exigences de qualité du back-end (CRUD complets, sécurité, validation serveur). La sobriété n'est pas un obstacle : elle force des choix architecturaux clairs et une discipline de scope qui profitent à la qualité globale du code.

Le projet remplit les objectifs Green IT initialement fixés : poids moyen sous 15 Ko, score EcoIndex A, score Lighthouse Performance $\geq 97/100$, zéro dépendance JavaScript externe, zéro polices web. Le pipeline CI/CD GitHub Actions + Railway garantit que ces engagements restent vivants au-delà de la livraison.

Comme le rappelle Antoine de Saint-Exupéry dans la citation reprise par l'énoncé : « la perfection est atteinte non pas lorsqu'il n'y a plus rien à ajouter, mais lorsqu'il n'y a plus rien à retirer. » C'est le principe directeur qui a guidé chacune des décisions de conception et d'implémentation de LocalMarché.

9. Annexes

9.1 Liens utiles

- **Dépôt GitHub** : <https://github.com/maxime-gabouty/localmarche>
- **Site déployé** : <https://localmarche-production.up.railway.app/index.php>
- **Diagrammes UML** : dossier /docs/ du dépôt (cas d'utilisation, MLD, séquence)
- **Captures écran EcoIndex / Lighthouse** : dossier /docs/captures/

9.2 Comptes de démonstration

Tous les comptes du jeu de démonstration ont pour mot de passe : demo1234

Email	Rôle	Usage en démo
admin@localmarche.fr	admin	Modération users
pierre@ferme-bio.fr	producteur	Dashboard avec 4 produits
marie@verger.fr	producteur	Dashboard avec 2 produits
lucas.bernard@mail.fr	consommateur	Passage de commande
emma.petit@mail.fr	consommateur	Vue commandes passées

9.3 Statistiques du projet

- 27 fichiers PHP, ~2 000 lignes de code
- 1 fichier CSS de 245 lignes (6,4 Ko)
- 1 fichier JavaScript de 14 lignes (416 octets)
- 4 tables SQL, 8 index secondaires
- 3 diagrammes UML SVG
- 0 dépendance Composer ou npm
- 0 police externe, 0 image

9.4 Backlog complet

Le backlog défini en Partie 1 est livré intégralement en V1 :

ID	Fonctionnalité	Priorité	Difficulté	Statut
B01	Auth : inscription / connexion	Haute	Moyenne	✓ Livré
B02	CRUD profil utilisateur	Haute	Faible	✓ Livré
B03	CRUD produits (producteur)	Haute	Moyenne	✓ Livré
B04	Page liste producteurs + filtres	Haute	Moyenne	✓ Livré
B05	Page détail produit	Haute	Faible	✓ Livré
B06	Commande simplifiée (panier)	Moyenne	Haute	✓ Livré

ID	Fonctionnalité	Priorité	Difficulté	Statut
B07	Footer éco : poids page + CO ₂	Moyenne	Faible	✓ Livré
B08	Administration : modération	Faible	Faible	✓ Livré